Date _/_/_

What is Database.

DBMS is a collection of Data.

Types of DBMS

1. SQL 2. NoSQL

# Data :- Data is a collection of Raw facts.

Examp- Numbers, text etc.

# Database :- The organized collection of structured da
that is stored & accessed electronically
to manage more efficiently called DBMS.

Examp- student details
↳ Name, Rollno., Address, no., Age, gender

# Why Database came :-

Data store Technique.
↳ Books & file system. X Not uses
↳ Flat file system X not used
↳ Database :- Uses Present Time

# Flat file system :- when computer comes into the picture
to overcome the problems of books &
file system.

Date _/_/_

Advantage :-

- Backup
- Editing
- Remote Access
- Sharing & Portability.

Dis-Advantage flat file system :-

- Data retrieval Problem.
- Security Problem.
- Data redundency problem.
- Data integrity.
- Indexing problem.

Database Advantage :-

- Data retrieval.
- Security.
- Data redendency.
- Data integrity.
- indexing.

# What is DBMS (Database Management system) :-

DBMS stand for Database Management system.

It is the software designed to create, manage & intract with dadabase.

Date _/_/_

Types of DBMS :-

- RDBMS (Rational DBMS)
- No SQL DBMS (
- HDBMS
- NDBMS
- OODBMS

RDBMS - (Relational Database Management System) -

Store Data in tabular Format (raw and column).

examp- Oracle, My SQL, Postgre SQL etc.

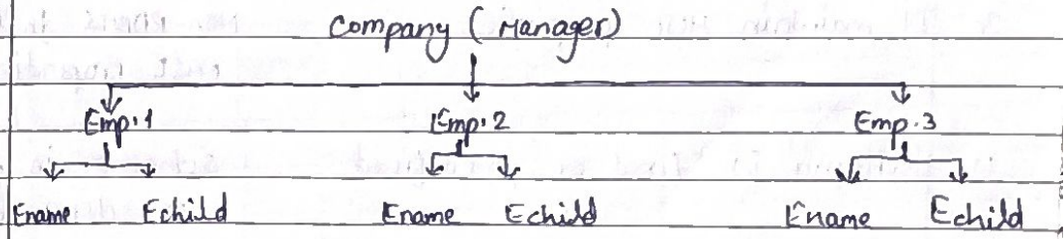
NoSQL DBMS -

Store data in unstructured format.

examp- Mongo DB, Redis etc.

HDBMS - (Hierarchical Database Management System) -

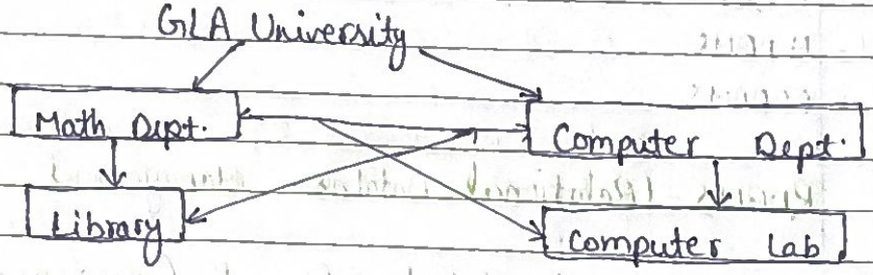
Store data like tree (organizational structure).



Date _/_/_

# NDBMS (Network & Database Management System) :-

Store Data in graphical structure (where nodes are directly connected to each other.)




# RDBMS b/w Non-RBDMS Diff:

| RDBMS | Non - RDBMS |
|---|---|
| 1- Store data in tabular format | Store data in key-value document, graph and more. |
| 2. Support structure data with Types (integer, string, date etc) | Support structured semi-structured and unstructured data (JSON, XML, binary etc) |
| 3- It maintain ACID properties | Non-RDBMS follows the BASE Properties. |
| 4. schema is fixed or predefined | schema is flexible or schema-less. |
| 5- It came early 1970. | Introduced on late 90's (1998). |

Date _/_/_

| Example - RDBMS | Example - Non-RDBMS |
|---|---|
| Oracle | Mongo DB |
| MySQL | Redis |
| Postgre SQL | Neo4j |
| Microsoft, SQL Server | Amazon DynamoDB |
| and More | Couchbase & More. |

Q- What is relationship ? full explanation.

RDBMS allow us to cearte Relationship among the diffrent tables.

# One to One (1:1)

if each record of table enter-related to only each record of another Table.

exampe-

Person

| ID | Name |
|---|---|
| 100 | XYZ |
| 200 | ABC |

Passport

| ID | Passport (PD) | Pass No. |
|---|---|---|
| 100 | 1001 | H13508 |
| 200 | 1002 | H 24350 |

# One to many (1:M)

if one record of a table enter-realated to multiple records of Another Table.

Example.

Customer

| Customer ID | Name |
|---|---|
| 1 | XYZ |
| 2 | ABC |

Order

| Order ID | Custord | Date |
|---|---|---|
| 101 | 1 | 6 Aug |
| 102 | 1 | 17 Jan |
| 103 | 2 | 20 Apr |

Date _/_/_

# Many to Many Relationship –

if multiple records of one Table are related to multiple records of Another Table.

examp

Student

| Roll | Name |
|---|---|
| 101 | XYZ |
| 102 | ABC |
| 103 | PQR |

Course

| C-id | C-name | C-fee |
|---|---|---|
| 3204 | Python | 2000 |
| 3448 | Java | 1800 |
| 5340 | React | 1900 |
| 7480 | Tailwind | 2300 |

# What is schema ? full explanation ?

Schema is nothing but blueprint that defines the structure of databasE. it includes the table, column, datatypes, relationship & constrains.

Note – Shema tells how data stored and related.

# Classification of schema.

1. Physical schema – Defines how data is stored on storage media.

2. Logical schema – Define the structure of Database in terms of table, column & relationship.

3- View schema – Defines how data is presented to users including access control & user specific view.

Relation – Table
Tuple – Raw
Column – Attribute

Date _/_/_

# What is Key ? Full explanation.

key is the data item in RDBMS. which exclusively indenfy the records of table.

# Primary key –:

Primary key is the key field which uniqly Identify each records of Table.

# Candidate key –: Candidate key are the key field which can be uniqly identify the records of table.

# Alternate key –: In a relation only one candidate can become primary key. the remaining candidate are called alternate key.

# Composite key –: Whenever more than one column combinally identify each record of Table is called composite key.

# Foreign key –: Foreign key is the field which uniqly identify the records of Another Table.

# Unique key –: Unique key is the key field which are also used to identify unique records from a table.

Date _/_/_

## What is ER model ? Full explanation

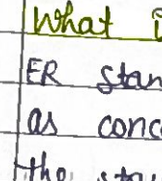
ER stand for entity relationship model it is also known as conceptual framework used to describe and design the structure of a database.

Before understanding the ER. model it is very important to know the diagram.

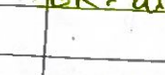
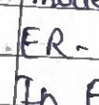
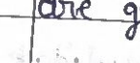
**ER. diagram** : The graphical representation of relationship of table is called ER diagram and the model through which we describe the relationship called ER. model.

In ER. diagram the logical structure of object (Table) are group together called relationship.

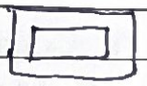
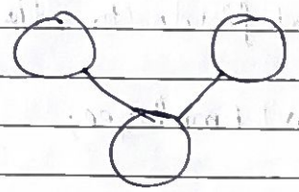
Note ER. diagram Relationship:




# while constant a ER. diagram we have to use some graphical symbols:

-  ⟹ Entity
-  ⟹ Attribute
-  ⟹ Relationship
-  ⟹ Primary key

Date _/_/_

-  ⟹ weak entity.
-  ⟹ weak entity relationship.
-  ⟹ Multivalued attribute
-  ⟹ composite attribute

# SQL – (Structed Query language)

- SQL is the standard language used to interact wit relational database.
- Allow users to store, retrieve, update and manage data efficiently through simple commands.

# SQL Command –

| DDL | DML | DCL | TCL | TQL |
|---|---|---|---|---|
| Create | Insert | | commit | |
| Drop | update | Revoke | Roll back | |
| ulter | delete | | save point | |
| Truncate | | | | |

# DDL – Data definition language is a category of SQL Command used to define and manage the structur of a database.

# Create Table – Used to creat a new tabl specifying

Date _/_/_

is name, column, data type.

## SQL Query keywords & Clauses

1- SELECT

- Used to choose which column/ fields to display.
- Select EmpName, Salary from Employee;
- Shows only EmpName and salary.

2. DISTINCT

- Remove duplicate values.
- Select Distinct Dept ID from Employee;
- List each department once even if many employee belongs to it.

3- where.

- Filter rows before displaying.
- Select * from employee where salary > 4000.
- Shows only employee with salary above 40000.

4 Between.
Used for range queries (inclusive)

Date _/_/_

- Select * from employee where salary Between 30000 and 50000.
- Salary ≥ 30000 and ≤ 50000.

5 AND/OR

- Combine Multiple Conditions.
- Select * from employee where Dept ID = 10 and Salary > 30000.
- Both condition must be true.
Select * from Employee where dept ID = 10 OR Dept ID = 20;
- Either condition is enough.

6- IN-
Matches a value from a list.
Select * from ~~student~~ (Employee) where Dept ID (10,20,30);
Employee must be in Dept 10, 20 or 30

7. IS NULL.
Check if a a Value is Null.
Select * from employee where ManagerID IS NULL.
Find employees without a manager.

Date _ / _ / _

8. LIKE
Used for Pattern Matching (wildcard % and _).
Select * from Employee where EmpName Like 'A%';

9. ORDER BY

Sorts output ascending (ASC default) or descending (DESC).

. Select * from Employee ORDER BY Salary DESC;

→ Employee shorted by salary highest.

10. Group by

Group rows by column(s) for a aggregation.

Select Dept ID, Avg (salary) from Employee Group by Dept ID.

° Average Salary Per Department.

11- Having

Filters group (used with Group by).

Select Dept ID, Count (*) from Employee Group by Dept ID
Having Count (*) > 5;

→ Only departments with more than 5 employees.

Date _ / _ / _

Subquery
A query inside another query.

Select EmpName From Employee where salary > (select
Avg (salary) From Employee);

→ Employees earning above Average salary.

Diffrence from other COUNT's;

Count (*) - Counts all rows (NULL included)
Count (column_name) → count only non-Null values
in that column.

→ Pattern Matching in SQL

1- Like operator
Used in where clause for matching text pat
wildcards:

% → Matches any Number of characters (inclu
_ → Matches exactly one character.

Date _/_/_

Q- What is EER Model? Full Explanation-

Ans- EER model stands for ~~the~~ enhance or extension entity Relationship Model. It is an extension of ER-model in compare of ER-model, this EER-model Makes complex real-world relationship more easy due to its additional concept like - "Generlization, specialization, aggeragation union sub-class and super class.

EER-model is a ~~design~~ high level data model which provides following features:-

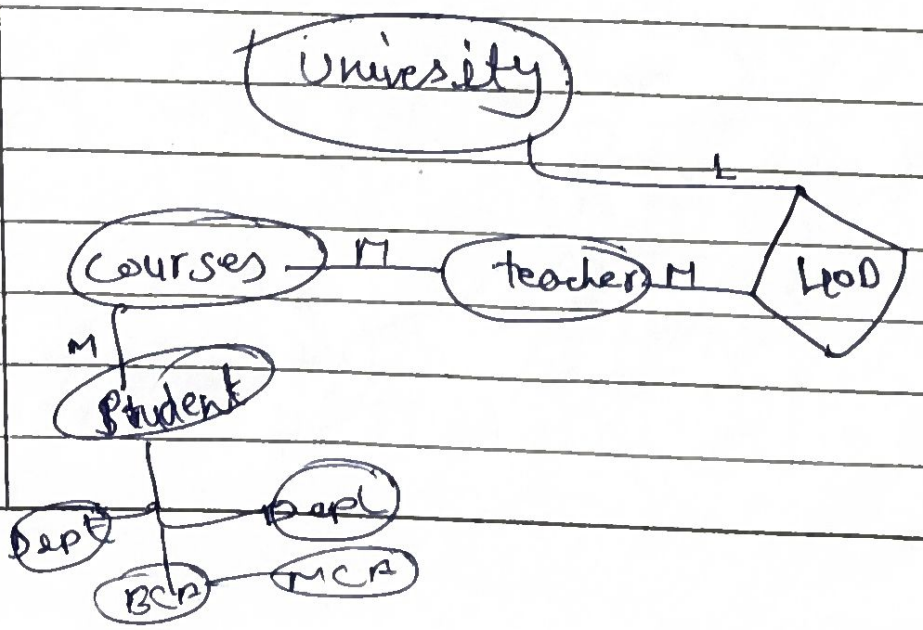
1- EER create a design more accurate to database syst.

2- It includes all the modeling features of ER-model.

3- EER model is widely used to design complex database in large Application.

Example- Create a database for university management system.

1) Person is a general entity
2) student and teacher are specialization or Person.
3) Department aggeregation. course and Dept.



Date _/_/_

Q- What is codes Rule!? Full Explaination→
The Rules are called codes value because they were included by "Dr. Edger Frankled Codd".
He was a British computer scientist who is considered the father of RDBMS

# Below is the summary of codd's 12 rules-

① Code's 0th rule - All the system must supports relational database features it is also known as system rule.

② Code's 1st rule - According to this rule all the data must be stored in tabular formed it is also called information rule.

③ codes 2nd rule - In this rule a table must contain a table (column) which can pick a record uniquely it is also known as guaranteed Access rule.

④ codes 3rd rule -

Date _/_/_

Q- what is the data abstraction in ER Model? full Explaination.

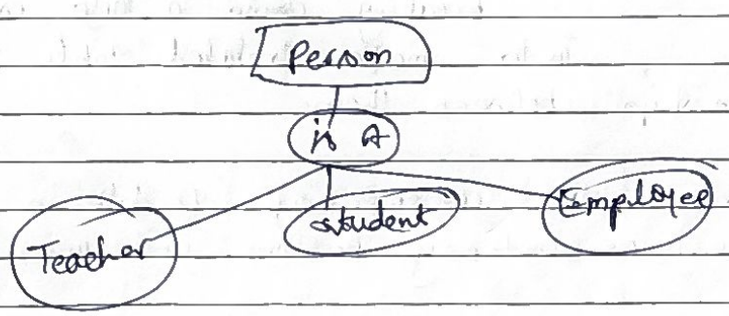
Ans- Data abstraction is used to represent complex relation ship between entities.

There are three types of abstraction

(i) Generalization
(ii) Aggeregation
(iii) specialization

① Generalization ⇒ Process of converting low-level entity to High level entity is called generalizati

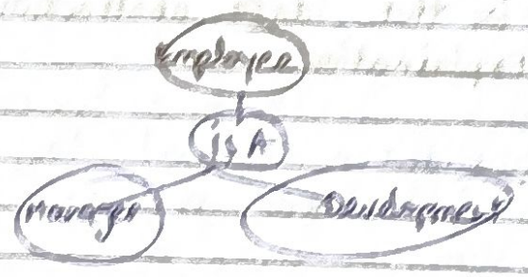
example- suppose we have three entities, teacher Employee, student



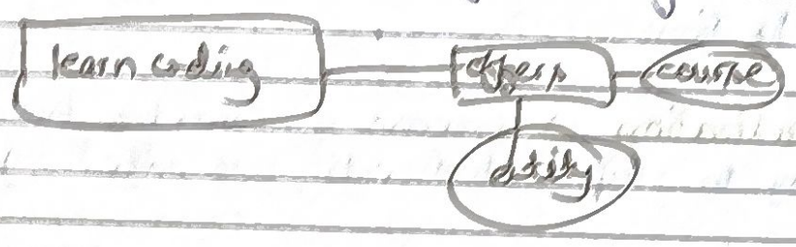
Date _ / _ / _

# Specialization → it is opposite of generalization it create sub-entity from high level entity

example - Employee is a high level entity of specialize it into sub entities.




# Aggregation → Aggregation is a process when relation between two entity is treated as a single entity.




# Normalization → Normalization is a method to breaking down a large, complex table into smaller related table and define relationship between them.

It reduce the redundancy of data in a table also remove inconsistency problem and unwanted risk.

Date _ / _ / _

# Advantage.
- maintain data integrity
- reduce the size of table
- Avoid null value

Types: ① 1 NF ② 2 NF ③ 2nf ④ BCNF ⑤ 4 NF ⑥ 5 NF

# BCNF → left hand side all is to super ck. ~~and right table prime~~

# 3nf → Right hand side prime attribute and left hand side ck.

# 2nf → left hand side proper subset of any ck and right hand a non-prime attribute.

# 1NF → Atomic value.

$R = (AB, EF)$ $\{AB \to C, C \to DE, E \to F, F \to E\}$

Step-1

$AB^+ = ABCDEF$ (AB, FB, EB, CB) Step-2 Prime attribute

$FB^+ = FBACDE$ Non Prime (D)

$EB^+ = EFA$

$CB^+ = CBDEFA$

Step-3

| | AB → C | C → DE | E → F | F → A |
|---|---|---|---|---|
| BCNF | ✓ | ✗ | ✗ | ✗ |
| 3NF | ✓ | ✗ | ✓ | ✓ |
| 2NF | ✓ | ✗ | ✓ | ✓ |
| 1NF | ✓ | ✓ | ✓ | ✓ |

Date _/_/_

# What is data ⇒ Data is a collection of Raw facts Data are characteristics Collected through observation and can be quality ~~data~~ tive any facts and figures about and entity is called data.

# What is information ⇒ Data becomes information when analyzed and placed in context, providing a base for understanding decision-making and further analysis Process data is called information.

# What is data base ⇒

A Database is a structured collection of data facilating easy access, management and updates generally stored and accessed deblectronically from a computer system.

# Data base Management System ⇒

It is software that helps in storing, managing retrieving and manipulating data in databases

# What is DDLC

DDLC is structured process ~~o~~gigning, creating and maintaining a database system it is similar to the Software development life cycle but focused on databases

Date _/_/_

| | DBMS | File system |
|---|---|---|
| ① | DBMS is a collection of data in dbms user is not required to write the procedures | The file system is a collection of data in this system the user has to write the procedures for manging the database |
| ② | In DBMS due to centrilized approach data sharing is easy | In file system Data is distributed in many files and it may be of different format so it is not easy to share data |
| ③ | Dbms gives an abstract view of data that fide the details | The file system Provides the details of the data representation and storage of data |
| ④ | In DBMS data redendancy Problem is not found. | In file system redundancy problem is found. |
| ⑤ | Data Inconsistency dass not exist. | Data Inconsistency exist |
| ⑥ | DBMS database structure is complex to design | The file system approach has a simple ~~structure~~ structure |

Date _/_/_

# Dependency Preserving ⇒

Let R(ABCD) with function dependies
{A → B  B → C , C → D , D → B}
R is decomposed into $R_1(AB)$, $R_2(BC)$, $R_3(BD)$

| $R_1(AB)$ | $R_2(BC)$ | $R_3(BD)$ |
|---|---|---|
| A → B ✓ | B → C ✓ | B → D ✓ |
| B → A | C → B ✓ | D → B ✓ |

A → B ✓
B → C ✓
C → B ✓
B → D ✓
D → B ✓

Q- R(A, B, C, D)
A → B
B → C
C → D

Decomposition

| $R_1(A,B)$ | $R_2(B,C)$ | $R_3(C,D)$ |
|---|---|---|
| A → B ✓ | B → C ✓ | C → D ✓ |
| B → A ✗ | C → B ✓ | D → C |

Q- R(W X Y Z A)

$f = \{W \to X,\ X \to Y,\ Y \to Z,\ Z \to A\}$

| $R_1(WX)$ | $R_2(XY)$ | $R_3(YZ)$ | $R_4(ZA)$ |
|---|---|---|---|
| W → X ✓ | X → Y ✓ | Y → Z ✓ | Z → A ✓ |
| X → W ✗ | Y → X ✗ | Z → Y ✗ | A → Z ✗ |

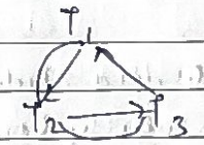
W → X ✓
X → Y ✓
Y → Z ✓
Z → A ✓

$R_1 \cup R_2 \cup R_3 = R$

Date _/_/_

# Serializibility ⇒

| $T_1$ | $T_2$ | $T_3$ |
|---|---|---|
| R(A) | | |
| W(A) | | |
| | R(A) | |
| | W(A) | |
| | | R(A) |
| | | W(A) |




# Conflict Serializibility

S

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| | R(A) |
| | W(A) |
| R(B) | |

S'

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| R(B) | |
| | R(A) |
| | W(A) |

| R(A) | (R(A) Non conflict pair |
|---|---|
| R(A) | W(A) |
| W(A) | W(A) |
| W(A) | R(A) |

conflic pair

non conflic pair

| | |
|---|---|
| R(B) | R(A) |
| W(B) | R(A) |
| R(B) | W(A) |
| W(A) | W(B) |

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| | R(A) |
| R(B) | |
| | W(A) |

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| R(B) | |
| | R(A) |
| | W(A) |

S ≡ S' are equivalent

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| R(B) | |
| | R(A) |
| | W(A) |

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| R(B) | |
| | R(A) |
| | W(A) |

Date _/_/_

| | | |
|---|---|---|
| (7) | Accessing database is easier | Accessing database is comparatively Difficult |
| (8) | ~~In DBMS data Independence exits~~ Integrity Constraints are easy to Apply | Integrity constraint are diffecult to implement in file system. |

Flat file system

What is DBMS Architecture ? Full Explaination

The design of a DBMS depends on its architecture and it can be centralized or decentralized. It can defines how users intract with a database system and how data is managed and accessed.

| # | three tier |
|---|---|
| 1 | 1- tier |
| 2 | 2- tier |
| 3 | 3- tier |

Date _/_/_

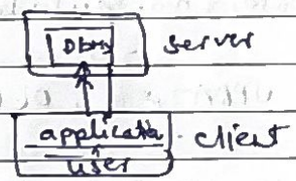
1 tier ⇒

The client server and database are all on the same m/c. In this architecture user directly intract with the database.

Database designer and programme normally prefer to use single tier architecture.

2 tier ⇒

in this architecture must have an Application through which the database can be access. there client directly intract with the server




3 tier ⇒

in this architecture client can't directly intract with server. It seprates the database, application logic, and use interface into three distinct layer.

Q- Data abstraction in ER model

Data abstraction is used to represent complex relationship between entities.

There are three type of Abstraction

①

Date _/_/_

# Database language and Interfaces (DDL, DML, DCL)

Sql stand for 'structured query language every relational database software introct with a language known as sql because It's a simple english like language which guidliner

DDL ⇒ (Data Definition language)

CREATE, DROP, TRUNCATE, ALTER, RENAME

DML ⇒ (Data manipulation language)

INSERT, UPDATE, DELETE

DCL ⇒ (Data control language)

GRANT, REVOKE

## Integrity Constraint over Relation

- Integrity Constraint are used to ensure accuracy and consistency of the data in a Relational database.
- Integrity Constraint are set of rules that the database is not permitted to violate.
- Constraints may apply to each attribute or they may apply to relationship between table.

Date _/_/_

example: A blood group must B 'A' or 'B' or 'AB' or 'O' only (can not any other value else)

① Types of

1. Domain Constraint
2. Entity Integrity Constraint
3. Referential Integrity Constraint
4. key constraint

## Domain Constraint ⇒ defines the domain or the valid set of values for an attributes.

The data type of domain includes string, character integer, time, date, currency etc. The value of the attributes must be availble in the corresponding domain

| SID | NA. | SEM. | Age |
|---|---|---|---|
| 11 | M | 3 | 20 |
| 12 | X | 4 | 30 |
| 13 | Y | 5 | A |

Domain

② The Entity Integrity Constraint states that primary key value can not be null.

Date _/_/_

This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value then we can't identify those rows.

(3) A Referencial integrity Constraint is specified between two table.

- Referencial Integrity Constraint If a forgein key in Table 1 refer to the primary key of Table 2
- Referencial Integrity Constraint is enforced when a forgein key refernce the primary key of a table.

# Transaction

Date _/_/_

# Transaction ⇒ It is a set of operation used to perform a logical unit of work.

→ A transaction generally represent change in database

# ACID Properties

Atomicity Consistency Isolation Durability

(1) Atomicity ⇒ It states that all operations of the transaction take place at once if not the transaction is aborted there is no midway i.e the transaction cannot occur partially Each transaction is treated as one unit and either run to completion or is not executed at all.

| A=600 | B=300 |
|---|---|
| $T_1$ | $T_2$ |
| read (A) | read (B) |
| A = A - 100 | B = B + 100 |
| write (A) | write (B) |
| A = 500 | B = 400 |

If transaction failed so transaction will be rollback

(2) Consistency ⇒ The integrity constraints are maintained so that the database is consistant before and after the transaction.

| $T_1$ | $T_2$ |
|---|---|
| read (A) | read (B) |
| A = A - 100 | B = B + 100 |
| write (A) | write B |
| A = 500 | B = 400 |

Date _/_/_

# Isolation ⇒ It shows that the data which is used at the time a execution of a transaction cannot be a used by the second transaction until the first one is completed.

| $T_1$ | $T_2$ | $T_3$ | $T_4$ |
|---|---|---|---|
| A - B | C - D | E - F | G - H |

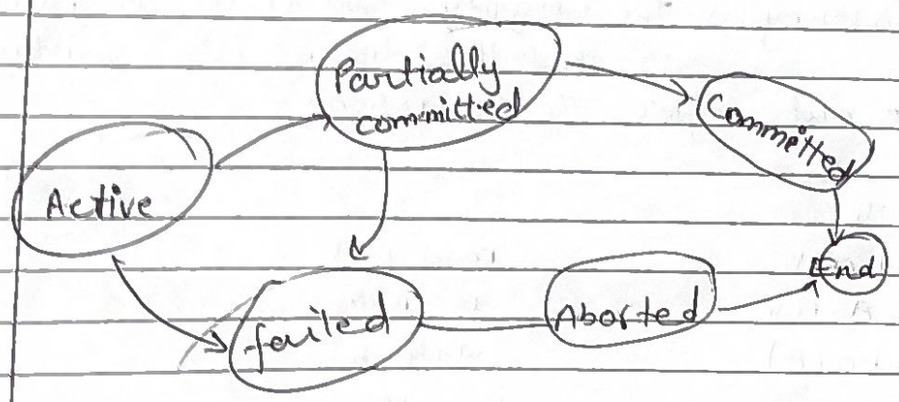
After one transaction completed execute will be perform the second transaction.

# Durability :-

The durability property is used to indicate the performance of the database's consistant state. It states that the transaction made the permanent changes.

# States of transaction :-

In a database the transaction can be in one of the following states.




Date _/_/_

# Serializibility of schedule

# Schedule :- when several transaction are executed concurrently then the order of execution of various transaction is known as a schedule. The concept of serializibility of schedules is used to identify which schedules are corrected when transaction executions have interleaving of their operations in the schedules.

Types of schedules

| Serial Schedule | Non-serial schedule | conflict schedule | view schedule |
|---|---|---|---|

Serial schedule ⇒ is a type of schedule where one transaction is executed completely before starting another transaction. In the serial schedule when the first transaction completes its cycle then the next transaction is executed.

$T_1$ $T_2$

R(A)

Non-serial Schedule ⇒ A non serial schedule is a schedule in which operations of multiple transaction are interleaved (Mixed)

W→R
W→W

Date _/_/_

It means transaction do not run one after another completely

| $T_1$ | $T_2$ |
|---|---|
| Read (A) | |
| write (A) | |
| | Read (B) |
| | write (B) |

# Irrecoverable Schedule

- A transaction reads data written by another transaction before it commits
- later the transaction that wrote the data fails.
- The second transaction cannot be rolled back safely.

LO
A=A-5
B

Example

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| A=A-5 | |
| W(A) 5 | |
| | R(A) 5 |
| | A=A-2 |
| | W(A) 3 |
| | commit |
| R(B) | |
| fail | |

# Recoverable Schedule →

A schedule is recoverable when no transaction commit based on dirty data.

Date _/_/_

- If transaction $T_2$ reads data written by $T_1$
- Then $T_2$ will commit only after $T_1$ commit.

# Serializibility ⇒

A schedule is serializable if its final result is the same as the result of some serial execution of the transaction. Serializability ensures consistant in concurrent transaction execution.

Example

S

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| | R(A) |
| | W(A) |
| W(A) | |
| | R(B) |
| | W(B) |

S

| $T_1$ | $T_2$ | $T_3$ |
|---|---|---|
| ~~R(A)~~ | R(A) | |
| | | R(A) |
| | | W(A) |
| | W(A) | |
| | W(B) | |

$T_1 \rightleftarrows T_2$

$T_1 - T_2 - T_3$
$T_1 - T_3 - T_2$
$T_2 - T_3 - T_1$
$T_2 - T_1 - T_3$
$T_3 - T_2 - T_1$
$T_3 - T_1 - T_2$

# Conflict equivalent ⇒

S

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| | R(A) |
| | W(A) |
| R(B) | |

S'

| $T_1$ | $T_2$ |
|---|---|
| R(A) | |
| W(A) | |
| R(B) | |

| | |
|---|---|
| R(A) | R(A) } Non conflict pair |
| R(A) | W(A) } |
| W(A) | R(A) } Conflict pair |
| W(A) | W(A) } |
| R(B) | R(A) W(A) W(B) } |
| W(A) | R(A) |
| R(B) | W(A) Non Conflict |

Date _/_/_

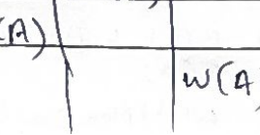
# Conflict serializability :-

A schedule is conflict Serializable if it can be changed into a serial schedule by swapping only those operations that do not conflict.

Example

$R(A) \rightarrow W(A)$
$W(A) \rightarrow R(A)$
$W(A) \rightarrow W(A)$

| $T_1$ | $T_2$ | $T_3$ |
|---|---|---|
| R(A) | | |
| | | R(B) |
| | | R(A) |
| | R(B) | |
| | R(C) | |
| | | W(B) |
| | W(C) | |
| R(C) | | |
| W(A) | | |
| W(C) | | |




No loop / cycle

Conflict serializable

$T_2 - T_3 - T_1$

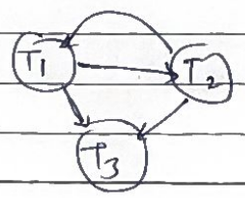
$T_1 - T_2 - T_3$
$T_1 - T_3 - T_2$
$T_2 - T_1 - T_3$
$T_2 - T_3 - T_1$
$T_3 - T_2 - T_1$
$T_3 - T_1 - T_2$

# View serializibility

Q- check weather schedule is conflict serializability or not ?

S

| $T_1$ | $T_2$ | $T_3$ |
|---|---|---|
| R(A) | | |
| | W(A) | |
| W(A) | | |
| | | W(A) |




Non-conflict serializable

S1

Date _/_/_

| $T_1$ | $T_2$ | $T_3$ |
|---|---|---|
| R(A) | | |
| | W(A) | |
| W(A) | | |
| | | W(A) |

# Recovery from Transaction Failure

Recovery in DBMS is the process of restoring the database to a correct and consistant state after a transaction failure or system crash.

If a transaction fails, DBMS ensures that

- Incomplete transaction are undone.
- Complete transaction are not lost.

# Causes of transaction failure :

1. Deadlock
2. System crash
3. logical error
4. Hardware or software failure

# Shared - Exclusive locking

Share lock : if transaction locked data item in shared mode, then allowed to read only

Exclusive : if transaction locked data item in Exclusive mode then allowed to read only.

Date _S/_/_

| grant S \ Request | Yes | No |
|---|---|---|
| Yes | Yes | No |
| No | No | No |

# 2 Phase locking

Growing locking Phase ⇒ locks are required and no locks are released

Shrinking Phase ⇒ locks are released and no locks are required.

| $T_1$ | $T_2$ |
|---|---|
| lock S(A) | |
| | lock S(A) |
| lock S(B) | |
| Unlock A | |
| | lock X(B) |
| Unlock(B) | |
| | Unlock(A) |
| | Unlock(B) |

# Strict 2PL ⇒ It should satisfy the basic 2PL and all exclusive locks should hold until commit/Abort

# Rigous 2PL ⇒ It should satisfy the basic 2PL and all shared, Exclusive locks should hold until commit/Abort

| $T_1$ | $T_2$ |
|---|---|
| X(A) | |
| R(A) | |
| W(A) | |
| | S(A) |
| | R(A) |